

Guide de développement pour projets app, web et PWA

Ce guide sert de fil conducteur pour modifier ou étendre un projet. Il s'appuie sur l'architecture du dépôt Manounou et sur les bonnes pratiques courantes. Son objectif est de garder un code clair et modulaire tout en avançant vite.

Philosophie

- Restez simple. Chaque fichier doit avoir une responsabilité précise.
- Séparez l'affichage, les données et la logique. Le **modèle** décrit les données, la **vue** affiche l'interface et le **gestionnaire** (ViewModel ou contrôleur) gère l'état et interagit avec les services.
- Évitez les solutions lourdes ou abstraites sans besoin. Recherchez la clarté avant tout.

Structure recommandée

Organisez le dépôt comme suit :

- `App/` : point d'entrée et composition globale (navigation, injection de services).
- `Features/<NomDeFonctionnalité>/` : un dossier par fonction (ex. Articles, Calendrier, Enfants).
Chaque dossier contient :
 - `Model/` : structures de données et types.
 - `ViewModels/` : classes ou fonctions qui exposent l'état à la vue, avec un constructeur qui accepte les services nécessaires.
 - `Views/` : vues SwiftUI, composants React ou pages HTML qui restent "bêtes" ; elles observent les ViewModels et déclenchent des actions.
 - `Services/` : protocoles et implémentations pour récupérer ou stocker les données (API, base locale, fichiers).
 - `Shared/` : éléments réutilisables par plusieurs fonctionnalités (composants UI, helpers, styles, constantes).
 - `Resources/` : images, couleurs, polices et textes localisés.
 - `Tests/` : tests unitaires pour les ViewModels et tests d'interface.

Adoptez des conventions de nommage cohérentes : `FeatureNameView`, `FeatureNameViewModel`, `FeatureNameService`. Un fichier par type simplifie la recherche.

Ajouter une nouvelle fonctionnalité

1. **Définir le besoin** : écrivez en quelques lignes ce que l'utilisateur final veut faire. Identifiez les données, les actions et les cas d'erreur.
2. **Créer le dossier** `Features/<Nom>` avec les sous-dossiers décrits plus haut.

3. **Définir le modèle** : créez les types qui représentent les données. Ajoutez les protocoles de service (ex. `ArticleService`) et une implémentation qui fait les appels réseau ou la lecture de fichiers.
4. **Écrire le ViewModel** : déclarez les propriétés observables (`@Published` en Swift, `state` en React). Injectez les services via le constructeur. Incluez les méthodes pour charger, créer ou modifier les éléments.
5. **Construire l'interface** : composez des vues réutilisables et simples. Évitez les valeurs de taille fixes ; préférez les grilles flexibles et les contraintes `maxWidth: .infinity` ou des unités relatives (pour le web, utilisez `flex` et `grid`).
6. **Relier la navigation** : exposez la fonctionnalité dans l'écran principal (ex. via un onglet ou un lien). Passez le ViewModel en paramètre ou via un fournisseur de contexte.
7. **Écrire des tests** : testez le ViewModel avec un service simulé (mock). Vérifiez les états de chargement, succès et échec.
8. **Documenter** : mettez à jour le `README.md` ou un fichier `docs/` avec la description de la fonctionnalité et les API utilisées.

Modifier une fonctionnalité existante

1. **Identifier l'impact** : quel module sera touché ? Listez les fichiers concernés.
2. **Appliquer les changements** dans les services, ViewModels ou vues sans casser la séparation. Ne mettez pas de logique réseau dans la vue.
3. **Mettre à jour les tests** : adaptez les tests pour couvrir les nouveaux comportements. Exécutez la suite pour vérifier qu'elle reste verte.
4. **Réviser les dépendances** : si un nouveau service est ajouté, déclarez un protocole et injectez-le dans le constructeur des classes concernées.
5. **Nettoyer le code** : supprimez les imports et les ressources inutilisés. Renommez les éléments incohérents.

Services et injection de dépendances

- Déclarez toujours un **protocole** pour chaque service (`AuthService`, `EventsService`, etc.).
- Fournissez une **implémentation réelle** qui se connecte au réseau ou à la base et une **implémentation simulée** pour les tests.
- Passez ces services au ViewModel via un constructeur. Évitez de créer des instances directement dans la vue.

Contrôle de version et branches

- Travaillez dans des **branches** isolées. Utilisez des noms clairs : `feature/nom-fonction`, `bugfix/id-ticket`.
- Faites des **commits atomiques** ; chaque commit doit représenter un changement logique et être accompagné d'un message descriptif.
- Ouvrez une **pull request** lorsque vous êtes prêt. Demandez un regard extérieur si possible.
- Mettez à jour le `CHANGELOG.md` si vous en avez un.

Tests

- Écrivez des tests unitaires pour chaque ViewModel. Vérifiez que les méthodes exposent les bons états en fonction des réponses de service.
- Utilisez des simulations pour isoler les dépendances. N'attendez pas de vraie réponse réseau.
- Ajoutez des tests UI pour les flux critiques si la plateforme le permet (ex. XCTest pour iOS ou React Testing Library pour le web).
- Intégrez les tests dans un flux d'intégration continue (CI) afin de détecter les régressions tôt.

Qualité et accessibilité

- Respectez les guides de conception (couleurs, typographies, espacement) définis dans le dossier `Shared/`.
- Privilégiez les tailles de police adaptatives et les éléments accessibles (`DynamicType` en iOS, `rem` pour le web).
- Évitez de bloquer le fil principal. Utilisez `async/await` en Swift et `Promise` / `async` en JavaScript.
- Supprimez le code mort et les ressources non utilisées.

Cross-plateforme

- Pour un projet web ou PWA, adaptez cette structure : remplacez les dossiers `ViewModels` par `components` (React), et les services par des hooks ou classes.
- Utilisez un magasin d'état (ex. Redux, Vuex) pour partager l'état global. Gardez la même séparation entre modèle, vue et logique.
- Conservez la philosophie : modularité, injection de dépendances et tests.

Questions à se poser avant toute modification

- **Quelle est la valeur ajoutée de ce changement ?**
- **Quel module doit évoluer ?**
- **Comment tester ce nouveau comportement ?**
- **Qui risque d'être impacté ?**

En gardant ces points en tête, vous pourrez modifier votre code avec une approche lean et éviter la surinterprétation technique.
